

Assignment

Homework1

zchenlc

A ARIN5305 Homework Assignment

October 2, 2026

Task 1.2 Report: Can CodeBLEU and an AI Judge Measure Code Semantic Equivalence?

Background and setup

This report asks whether an automatically computed similarity score can replace human judgement about whether two programs do the same thing.

Four LeetCode problems were used, each contributing two code pairs. P-LLM pairs the course's correct solution with the code that DeepSeek-v4 Instant Chat regenerated from my Task 1.1 summary; that code passed all LeetCode tests. P-FLT pairs the same correct solution with the course's faulty program.

CodeBLEU 0.7.0 was run with `lang="java"`. Its four components (n-gram, weighted n-gram, syntax, data flow) carry equal weights of 0.25. The AI judge was DeepSeek-v4 Instant Chat in Instant mode, with Deep Think and Search off and one new conversation per pair. The template is given below.

```
You are a careful Java code reviewer. Two Java solutions to the same problem are given below as Solution A and Solution B. Neither one is labelled as correct or incorrect: judge only from the code itself and do not assume either one is right.
```

```
Please do all of the following:
```

1. In 1-3 sentences each, describe what Solution A and Solution B compute.
2. Decide whether A and B are functionally equivalent, i.e. whether they return exactly the same result for every valid input. Answer YES or NO.
3. If they are not equivalent, give one concrete input where they differ, together with the exact output of A and of B on that input, and explain why the behaviours differ.
4. List the 2-4 most important semantic differences (algorithm, data structures, boundary or edge-case handling, complexity). Ignore cosmetic differences such as variable names, comments, formatting or behaviour-preserving rewrites.
5. Give a semantic equivalence score between 0.00 and 1.00 (1.00 = identical behaviour on every valid input, 0.00 = completely different behaviour) and your confidence in this judgement (high / medium / low).

```
Do not run or simulate the code; reason about it directly.
```

```
Answer using exactly this structure:
```

- Behaviour of A:
- Behaviour of B:
- Equivalent: YES / NO
- Semantic equivalence score (0.00-1.00):
- Confidence: high / medium / low
- Key semantic differences:
- Counterexample (if not equivalent):

```
Solution A (Java):
```

```
{SOLUTION_A_JAVA}
```

```
Solution B (Java):
```

```
{SOLUTION_B_JAVA}
```

Question (1): CodeBLEU scores

On the same eight pairs, CodeBLEU produced two non-overlapping bands. The four P-FLT pairs fall between 0.592 and 0.978. The four P-LLM pairs reach only 0.270 to 0.378. Each reference program compared with itself returns 1.0000.

I suspect the high P-FLT scores follow from how those programs were made. Each faulty program changes one line or one branch of the reference, so the tokens, syntax tree and data-flow graph stay almost identical. The faulty version of `shortest-matching-substring` alters one character in a boundary condition, yet it still scores 0.9780 with a full-mark data-flow component.

The low P-LLM scores come from the opposite case. The LLM rewrote the implementation, and `shortest-matching-substring` and `minimum-swaps-to-sort-by-digit-sum` score only 0.3176 and 0.3783. This reverses my expectation.

The four components also tell me different things. Syntax favours structurally similar code, and the faulty pairs sit mostly between 0.83 and 0.97. The n-gram component drops furthest when data structures or variable names change, falling to 0.0938. Data flow did not seem to react to the one-character boundary change, and a program that behaves differently still reads 1.0000.

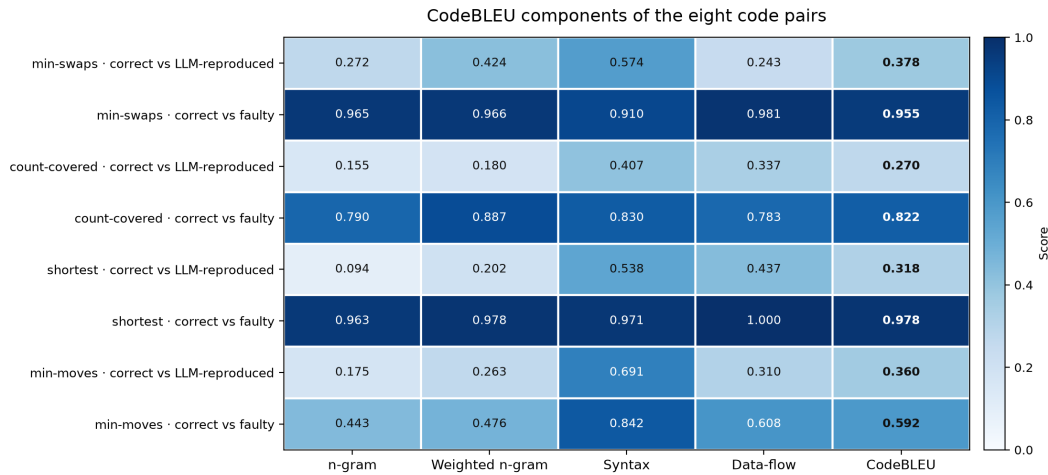


Figure 1: The four CodeBLEU components plus the total, for all eight code pairs. The syntax and data-flow components of the faulty pairs sit above 0.78 and row 6 reaches 1.000 on data flow, while the n-gram component of the LLM-reproduced code falls to 0.094.

Question (2): Key judgements of the AI judge

Where CodeBLEU returns a single number, the AI judge returns a structured answer. It states what each program computes, gives a YES or NO verdict, a semantic score between 0 and 1, a confidence level, and a counterexample input when the two programs are not equivalent.

Across the eight pairs the judge gave two YES and six NO verdicts. Both YES answers scored 1.00 and both were P-LLM pairs; the judge argued that the two programs are written differently but compute the same result. All four P-FLT pairs were judged NO, with a note

on where the program had been changed. Seven verdicts were marked high confidence. Only the P-LLM pair of `shortest-matching-substring` was marked medium.

Six answers supplied a counterexample input, which I record as given: `nums=[12,21,30]`, `s="aa"` with `p="a*a*a"`, and `classroom={"S.RL"}` with `energy=1`. I did not treat any of them as established.

The judge is also inconsistent. For the P-LLM pair of `minimum-moves-to-clean-the-classroom`, one reply first states “Equivalent: NO, score 0.20, confidence high” and later changes to “YES, 1.00, high” (Figure 3). I could not see a consistent scale behind the semantic scores either. Four faulty programs that each change one place receive 0.20, 0.20, 0.75 and 0.85, and one of them is judged NO while still scoring 0.85.

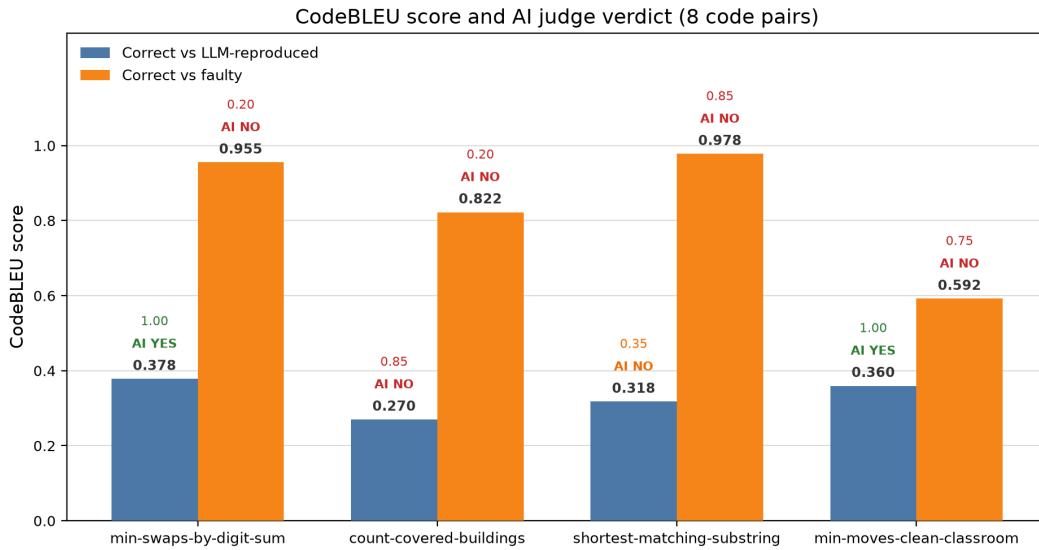


Figure 2: Bar height is the CodeBLEU score; the two labels above each bar give the AI judge verdict and its 0 to 1 semantic score. Blue is the LLM-reproduced pair, orange the faulty pair.

Wait, I need to double-check the pruning in B. The condition is

```
if (best[nx][ny][newMask] >= newEnergy) continue; . This means if we have seen this state with
energy >= newEnergy, we skip. But what if we have seen it with energy < newEnergy? Then we
update and add to queue. This is correct. However, there is a subtlety: BFS explores level by level.
The first time we reach a state, it's with the minimum steps. If we later reach the same state with
more steps but higher energy, we might update best and add to queue. But since we are doing
BFS, the later state has more steps, so even if it has higher energy, it might not lead to a shorter
overall path because we already spent more steps to get there. However, the goal is to minimize
total steps. If we reach the same state with more steps but higher energy, could that lead to a
shorter total path? No, because any continuation from that state will take the same number of
```

Figure 3: The re-thinking section of DeepSeek’s reply for the LLM-reproduced pair of `minimum-moves-to-clean-the-classroom`, where it revises the judgement it gave earlier in the same reply.

Question (3): Strengths and limitations of the two metrics

CodeBLEU is fast and cheap to run. Repeated runs on the same input usually agree, the eight pairs take seconds to score, and no network is needed. Because it parses syntax trees and data-flow graphs with tree-sitter, it stays closer to code structure than plain string comparison; one P-LLM pair has a literal similarity of 0.148 but a CodeBLEU score of 0.3176. The four components can also be inspected separately. On this small sample it looks usable as a first pass for bulk work such as plagiarism checks, clone detection or regression triage, though the evidence is thin.

Its score measures similarity of implementation, not sameness of behaviour. A one-character boundary change scores 0.9780, and a min/max rewrite scores 0.8224, while legitimate rewrites score only 0.3783 and 0.3176. I could not find a useful cut-off in these scores, and a reference program is required. One component is not even strictly deterministic: variable renaming relies on `list(set(...))` and therefore on the hash seed. Two of the eight pairs vary, by at most about 0.015.

The AI judge behaves differently. It tries to infer what the code is meant to do, needs no reference program, and explains its reasoning, so it handles rewrites and boundary faults that CodeBLEU misses. Its weaknesses are reliability and auditability. Answers are not reproducible, one reply can contradict itself, scores are not comparable across pairs, and every counterexample has to be confirmed.

In summary, on these eight pairs I do not think either method should be the single basis for a judgement. CodeBLEU can serve as a cheap, reproducible pre-filter. The AI judge can supply reasons, but those reasons need verification. Every difference it claims should be confirmed before it is cited, so that hallucination problems are avoided.